CS F213/MAC F212 OOP
Semester-I, 2026-2027

Project: Intelligent IT Service Management & Incident Resolution System

Project Philosophy

This project is **not a programming-language assignment**.

You are expected to approach the project as a software engineering team solving a real organizational problem.

You have the freedom to choose:

what to build, how to build it, which technologies to use, and how to architect the solution.

However, you must be able to justify your decisions.

The central question of the project is:

How can object-oriented thinking be used to design a maintainable, extensible and intelligent software system for managing IT incidents in a real organization?

Your final solution should demonstrate not only that the application works, but also that you can:

Analyze → Model → Design → Implement → Test → Evaluate → Defend

a real-world software system.

Project Overview

Modern organizations depend heavily on IT services such as cloud applications, databases, networks, authentication systems, enterprise applications, storage systems, communication platforms, and digital services.

When an IT service fails or behaves unexpectedly, employees or customers need to report the problem, and the organization's IT support team must identify, prioritize, assign, investigate, resolve, and close the incident.

In a large organization, hundreds or thousands of incidents may be generated every day. Managing these incidents manually can result in:

- Delayed response
- Incorrect prioritization
- Inefficient technician allocation
- SLA violations
- Poor visibility into service health
- Duplicate incident reports
- Incomplete incident history
- Difficulty identifying recurring problems

Your team is required to design and develop an **Intelligent IT Service Management & Incident Resolution System (I-ITSM)** that addresses these challenges.

The system should provide an integrated platform for ***incident reporting, classification, prioritization, technician assignment, SLA monitoring, resolution, notification, auditing, and reporting***.

Freedom of Technology Choice

This project is intentionally **technology-independent**.

Your team is free to choose the technologies, programming languages, frameworks, databases, platforms, and development tools that are most appropriate for your proposed solution.

You may develop:

Option A — Web Application

Examples:

- React / Angular / Vue + REST API
- Spring Boot
- Node.js / Express
- Django / FastAPI
- ASP.NET Core
- Laravel
- Other appropriate technologies

Option B — Mobile Application

Examples:

- Android / Kotlin
- Flutter / Dart
- React Native
- Swift / iOS
- Other appropriate mobile technologies

Option C — Full-Stack Application

A team may develop both:

- `Web/Mobile Frontend`
- `Backend/API Layer`
- `Business Logic`
- `Database`

Option D — Other Architecture

Teams may propose an alternative architecture such as:

- Cloud-native application
- Microservices
- Serverless application
- Desktop + backend application
- Progressive Web Application (PWA)

However, the proposed technology and architecture must be justified in the project report**.**

Technology selection is part of the project.

Students are expected to explain why the selected technologies are appropriate rather than simply choosing a technology because it is familiar.

Primary Objective

Develop an industry-oriented application that manages the complete lifecycle of IT incidents:
Report - Classify - Prioritize - Assign - Investigate - Resolve - Verify - Close
The application should use sound Object-Oriented Design principles and demonstrate how OOP concepts can be applied to a real-world software system.

Problem Scenario

Consider an organization with hundreds of employees and multiple IT services.
An employee encounters a problem: "The organization's payment application is unavailable and users are unable to complete transactions."
The employee reports the problem through the ITSM application.
The system should then:

1. Create an incident.
2. Classify the incident.
3. Identify the affected service.
4. Determine its business impact.
5. Calculate its priority.
6. Determine the applicable SLA.
7. Identify suitable technicians.
8. Assign the incident.
9. Notify relevant personnel.
10. Track investigation and resolution.
11. Monitor SLA compliance.
12. Record all important activities.
13. Close the incident after resolution.
14. Include the incident in relevant reports and analytics.

Minimum Viable Product (MVP)

Every team must implement at least the following:

User Management

- Employee
- Technician
- Administrator

Incident Management

- Create
- View
- Update
- Assign
- Resolve
- Close

Priority Management

- Impact
- Urgency
- Business criticality

Technician Assignment

At least two assignment strategies.

SLA Management

- SLA calculation
- Deadline tracking
- Violation detection

Notification

At least two notification mechanisms, simulated or real.

Audit

Complete incident history.

Search & Reporting

At least five meaningful search/filter criteria.

Intelligent Features

At least two intelligent/automated features.

Persistence

Database or equivalent persistent storage.

Major Users

The system should support at least three categories of users.

Employee / Customer

The user should be able to:

- Register/login
- Report an incident
- View submitted incidents
- Track incident status
- Add additional information
- Respond to support requests
- View resolution information
- Provide feedback after resolution

Technician / Support Engineer

The technician should be able to:

- View assigned incidents
- Accept or acknowledge an incident
- Investigate the incident
- Add investigation notes
- Update incident status
- Request additional information
- Provide resolution details
- Resolve an incident
- View current workload

Administrator / IT Manager

The administrator should be able to:

- Manage users
- Manage technicians

- Manage IT services
- Configure SLA policies
- Configure incident categories
- Configure assignment strategies
- Monitor incidents
- View reports
- View audit logs
- Monitor SLA violations

Teams may introduce additional roles where appropriate.

Incident Management

The system must allow users to create incidents.

Each incident should contain appropriate information such as:

- Incident ID
- Reporter
- Incident category
- Description
- Affected service
- Impact
- Urgency
- Priority
- Creation time
- Current status
- Assigned technician
- SLA
- Investigation details
- Resolution
- Closure information

Teams are free to decide the exact data model.

Incident Classification

Incidents should be classified into meaningful categories.

Possible categories include:

- Hardware
- Software
- Network
- Database
- Cloud
- Security
- Authentication
- Application
- Storage

Teams may define additional categories.

The system design should allow new categories to be introduced without requiring major changes to unrelated parts of the application.

Intelligent Priority Determination

The system should automatically determine incident priority using factors such as:

- Impact
- Urgency
- Business criticality
- Number of affected users
- Affected service

For example:

```
Priority Score = Impact × Urgency × Business Criticality
```

The system may classify incidents as:

```
P1 – Critical
P2 – High
P3 – Medium
P4 – Low
```

Teams may develop their own prioritization algorithm.

The algorithm and its rationale must be documented.

Advanced option

Teams may experiment with:

- Rule-based prioritization
- Weighted scoring
- Historical-data-based prioritization
- Machine-learning-assisted prioritization

AI/ML is optional**.** A well-designed rule-based system is perfectly acceptable.

Intelligent Technician Assignment

The system should recommend or assign an appropriate technician.

The assignment mechanism may consider:

- Technical skills
- Incident category
- Technician availability
- Current workload
- Experience
- Incident priority
- Previous experience with the affected service

Teams should implement at least **two assignment strategies**.

For example:

```
Skill-Based Assignment
Load-Based Assignment
Round-Robin Assignment
Priority-Based Assignment
Experience-Based Assignment
```

The system should allow the assignment strategy to be changed without redesigning the entire application.

Service Level Agreement (SLA) Management

Different IT services may have different Service Level Agreements.
For example:

Priority	Response Target	Resolution Target
P1	5 minutes	30 minutes
P2	15 minutes	2 hours
P3	1 hour	8 hours
P4	4 hours	24 hours

The system should:

- Identify the applicable SLA.
- Calculate deadlines.
- Track response time.
- Track resolution time.
- Identify SLA violations.
- Generate warnings before deadlines.
- Record SLA violations.

Teams may implement background processing for continuous SLA monitoring.

Incident Lifecycle

The system should support a well-defined incident lifecycle.
A possible lifecycle is:

```
NEW
 ↓
CLASSIFIED
 ↓
ASSIGNED
 ↓
ACKNOWLEDGED
 ↓
IN_PROGRESS
 ↓
RESOLVED
 ↓
VERIFIED
 ↓
CLOSED
```

The exact lifecycle may be modified based on the team's design.
However, the system must prevent inappropriate state transitions.
For example:

```
NEW → CLOSED
```

should not normally be allowed without appropriate intermediate processing.
This functionality should demonstrate appropriate object-oriented design.

Notification System

The application should notify relevant users when important events occur.
Examples:

- Incident created
- Incident assigned
- Priority changed
- Technician assigned
- SLA approaching
- SLA violated
- Incident resolved
- Incident closed

Possible notification mechanisms include:

- Email
- SMS
- Push notification
- In-app notification
- Dashboard notification

Actual external integration is optional. Simulation is acceptable.
The design should make it easy to introduce another notification mechanism later.

Incident Audit Trail

The system must maintain a history of important actions.
For example:

```
10:05 – INC1024 created by Employee E102
10:06 – Incident classified as APPLICATION
10:06 – Priority calculated as P1
10:07 – Assigned to Technician T205
10:12 – Technician acknowledged incident
10:20 – Investigation started
10:25 – Resolution provided
10:26 – Incident marked RESOLVED
10:40 – Incident CLOSED
```

Audit records should contain appropriate information such as:

- Timestamp
- User
- Action
- Incident ID
- Previous state
- New state
- Description

Search, Filtering and Reporting

The system should provide facilities for searching and filtering incidents based on criteria such as:

- Incident ID
- Date
- Status

- Priority
- Category
- Service
- Technician
- SLA status

The application should provide useful reports or dashboards.

Examples:

Incident Overview

```
Total Incidents       : 250
Open Incidents        : 42
Critical Incidents    : 8
Resolved Incidents    : 198
SLA Violations        : 10
```

Technician Workload

```
Technician    Active Incidents
--------------------------------
T101          5
T102          2
T103          7
T104          3
```

Service Analysis

```
Service              Incidents
--------------------------------
Database              35
Network               48
Authentication        21
Cloud Storage         16
Payment               12
```

Teams may develop additional analytics that provide meaningful operational insight.

Intelligent Features

Each team must implement **at least two intelligent/automated features**.

Possible features include:

A. Duplicate Incident Detection

Identify incidents that may represent the same underlying problem.

Example:

```
INC101 – Payment application unavailable
INC102 – Payment transaction failing
INC103 – Unable to make payment
```

The system may identify these as potentially related.

B. Incident Correlation

Identify incidents that may be associated with a common service or infrastructure component.

C. Automatic Escalation

If a critical incident is not acknowledged within the defined period:

```
Technician
     ↓
Team Lead
     ↓
System Administrator
```

```
     ↓
IT Manager
```

D. Smart Technician Recommendation

Recommend a technician based on skills, workload, experience and incident characteristics.

E. SLA Risk Detection

Identify incidents that are likely to violate their SLA.

F. Incident Categorization

Automatically suggest an incident category based on the incident description.
AI/ML may be used, but is not mandatory.

G. Suggested Resolution

For historical incidents, the system may suggest previously used resolutions for similar problems.

Object-Oriented Design Requirements

The project is an **OOP course project**; therefore, the quality of object-oriented design is a major evaluation criterion.
Teams must demonstrate appropriate use of:

Encapsulation

Data should be protected through appropriate access mechanisms.

Abstraction

Complex business operations should be represented through suitable abstractions.

Inheritance

Inheritance should be used only where an appropriate "is-a" relationship exists.

Polymorphism

Different implementations should be interchangeable where appropriate.

Interfaces

Interfaces should be used to define suitable contracts.

Composition

Objects should collaborate through composition where appropriate.

Association and Aggregation

Appropriate relationships between domain objects should be demonstrated.

Exception Handling

The system should handle exceptional and invalid situations appropriately.

Collections / Data Structures

Appropriate data structures should be selected based on application requirements.

Design Patterns

Each team must implement **at least two design patterns**.
Possible choices include:

Strategy Pattern

For:

- Priority calculation
- Technician assignment
- Notification selection

Factory Pattern

For:

- Incident creation
- Notification creation
- User creation

Observer Pattern

For:

- Notifications
- Audit events
- Incident status changes

State Pattern

For:

- Incident lifecycle

Repository Pattern

For:

- Data persistence

Adapter Pattern

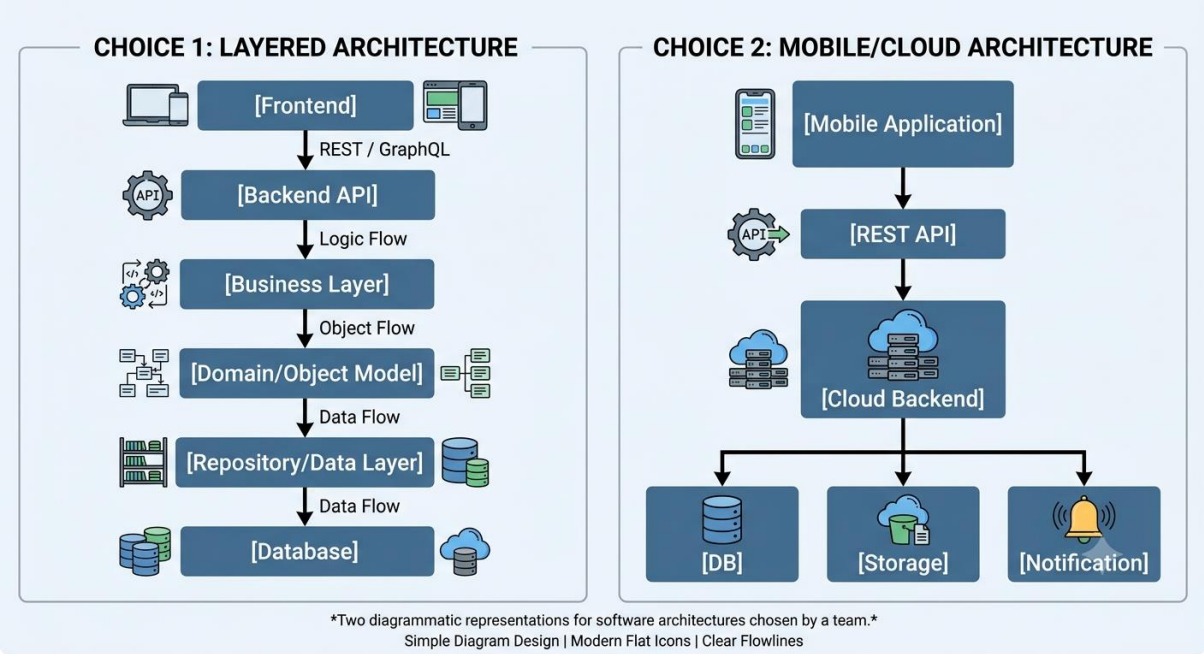
For:

- External notification/payment/cloud services

Teams may use other patterns if they can justify their selection.

Technology and Architecture Exploration

Students are encouraged to explore modern technologies.



Teams may explore:

- REST APIs
- GraphQL
- Cloud deployment

- Containerization
- Authentication
- OAuth/JWT
- Push notifications
- Cloud databases
- Serverless functions
- Microservices
- CI/CD
- Docker
- Kubernetes
- Observability
- AI/ML services

These are **optional**. Students should select technologies appropriate to their interest and project scope.

Technology Selection Report

Each team must justify its technology choices.

The report should answer:

1. Why did you choose this programming language?
2. Why did you choose this frontend technology?
3. Why did you choose this backend technology?
4. Why did you select this database?
5. Why did you choose this architecture?
6. What alternatives did you consider?
7. What are the advantages and limitations of your technology stack?
8. How does your technology support object-oriented design?
9. How would your system evolve if the number of users increased significantly?

Suggested Technology Combinations

These are examples only. Teams are **not necessarily required to use them**.

Frontend	Backend	Database
React	Node.js	MongoDB
React	Spring Boot	PostgreSQL
Angular	.NET	SQL Server
Flutter	FastAPI	PostgreSQL
React Native	Django	PostgreSQL
Android/Kotlin	Spring Boot	MySQL
Vue	Node.js	MongoDB

Other combinations are welcome.

Exception and Error Handling

The application should appropriately handle situations such as:

- Invalid login
- Unauthorized operation
- Invalid incident
- Invalid state transition
- Technician unavailable
- Technician workload exceeded
- Service not found
- Missing SLA configuration
- Duplicate incident
- Database failure
- Network failure
- Invalid user input

Teams should create appropriate custom exception/error handling mechanisms where required.

Security Requirements

Since this is an enterprise application, teams should consider basic security.
At minimum:

- Authentication
- Role-based authorization
- Password protection
- Input validation
- Secure API access
- Protection against unauthorized operations

Teams should document security considerations even if advanced security mechanisms are not implemented.

Persistence

The system must persist important information.
Teams may use:

- MySQL
- PostgreSQL
- MongoDB
- SQLite
- Firebase
- Cloud database
- Other suitable databases

The choice must be justified.
The system should preserve information such as:

- Users
- Services
- Incidents
- Technicians

- SLA policies
- Audit records
- Resolution information

Deliverables

Each team must submit the following.

1. Problem Analysis

Clearly explain:

- Problem
- Stakeholders
- Existing challenges
- Proposed solution
- Assumptions

2. Requirements Specification

Document:

- Functional requirements
- Non-functional requirements
- User roles
- Business rules

3. System Architecture

Provide an architecture diagram showing major components and their interactions.

4. UML Diagrams

At minimum:

Use Case Diagram

Showing actors and system functionality.

Class Diagram

Showing:

- Classes
- Attributes
- Methods
- Interfaces
- Inheritance
- Associations
- Composition/Aggregation

5. Technology Stack

Document all technologies, frameworks, libraries and services used.

6. Source Code

The complete source code must be submitted through an appropriate version-control repository.

Git/GitHub/GitLab/Bitbucket or an equivalent platform is recommended.

Share the link in the spreadsheet to be shared with you all.

7. Database Design

Provide:

- ER diagram where applicable
- Database schema

- Important relationships
- Sample data

8. Testing

Provide test cases covering:

- Normal operations
- Boundary conditions
- Invalid inputs
- Exceptions
- Authorization
- Business rules

At least **15 meaningful test cases** should be documented.

9. Project Demonstration

The teams should **record a video presentation** and must demonstrate at least three complete workflows. The recorded video to be uploaded to a shared folder (link will be shared)

For example:

Workflow 1

Employee reports → system prioritizes → technician assigned → notification generated.

Workflow 2

Technician investigates → SLA monitored → incident resolved → user notified.

Workflow 3

Multiple incidents reported → system identifies duplicates/related incidents → incidents correlated/escalated.

Team

Recommended team size:

3–5 students

Each member must have clearly identifiable responsibilities.

For example:

```
Student 1 → UI / UX
Student 2 → Backend / APIs
Student 3 → Domain Model / OOP
Student 4 → Database / Persistence
Student 5 → Testing / DevOps / Intelligent Features
```

However, **every student must understand the overall system.**

Innovation Challenge

Teams that want to go beyond the basic requirements may implement additional capabilities.

Possible extensions include:

- AI-assisted incident classification
- AI-based resolution recommendation
- Incident similarity detection
- Automated root-cause analysis
- Predictive SLA violation detection
- Service dependency graph
- Real-time notifications
- Mobile push notifications

- Voice-based incident reporting
- Chatbot-based incident reporting
- Cloud deployment
- Microservice architecture
- Event-driven architecture
- Real-time operational dashboard
- Analytics and visualization
- Automated incident escalation

These features are optional and should not compromise the quality of the core OOP design. Any team implementing any one of the above innovation challenges will be given additional weightage.

Evaluation Rubric

Criteria	Description	Marks
System Design & Architecture	It includes problem understanding and clear development of the prototype.	10
Functionality Implementation	Working implementation of mandatory modules (refer MVP).	60
User Experience	Value-added features (refer SLA, Security, option to add new features, intelligent features).	10
Technical Depth	Code quality, use of APIs (Google API, any other), integration with third-party services, scalability of design.	15
Testing & Validation	Unit testing, integration testing, bug resolution, handling edge cases (refer Exception Handling, Testing).	5
Innovation	Teams that went beyond the basic requirements and implemented additional capabilities. (refer Innovation Challenges)	10
Documentation & Presentation	Completeness of project report, clarity in documentation, and effectiveness of final recorded presentation (demo).	10
Total		120

Important Note:

- Students to work in groups (which should have been formed). The evaluation will be done for the group and all the members will get the same score. It will be up to the

members to have fair share of contribution for the successful implementation/development of the project. Arguments or feedback on individual member's contribution will not be entertained. If someone is not giving valuable contribution, then it depends upon other members to decide how to engage with the non-performing member. IC will not indulge.

- You are free to design the GUI for the above project as per your creativity. Appropriate links should be provided to transit from one option to another. Difference in GUI need to be maintained for different users. Feel free to make your assumptions in order to make the implementation better. Not everything can be mentioned in the description.
- You are not expected to develop from scratch. You can feel free to make use of available APIs or packages, AI tools or techniques, etc. The idea is to expose you to project development experience. High use of modern advanced technology, platform, paradigm will definitely attract high grades.
- Tentatively evaluation will be 120 marks as mentioned in the rubrics, which can be scaled up or down, as required to suit the weightage of this evaluation component.
- The marking will purely depend upon the evaluators and their evaluation will be final. No arguments in this regard will be entertained and will attract negative marking as per IC's discretion. In case of unethical practices, if observed, or brought to my notice, then without any discussion (as cannot engage in forensic investigations nor act as a moral police) all the groups involved will be awarded 0 marks.
- **Each group needs to make a PDF report (one per group) and record a video (demo of working application) presentation (15-20 mins)** of the developed application involving all the group members and upload it to a shared folder (link to upload will be shared later). The video file should be named by the Group No. **Supposing your Group No. is 1, then file should be name as 1_OOP_S1_2025-2026_Project.** While recording the demo (working application), each member should introduce himself/herself and explain the functioning of any module(s).
- The **last date to upload the recorded video presentation and the report is on or before 22.11.2026.** This is hard deadline and no request to extend the deadline will be entertained for operational/official limitations.
- **Do a good work so that you will be able to reflect in your resume and feel proud of your efforts. The best project(s) will be recommended to GAD to be adopted by Maintenance Division/CCIT of the Campus.**

Best Wishes